Problem set 2 Aashna Sibal

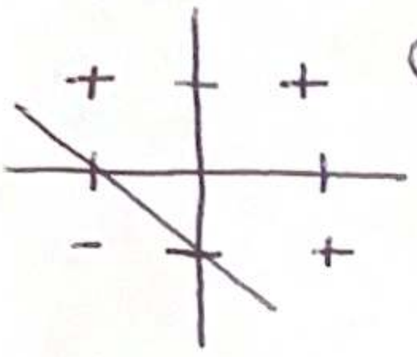
1) a) OR

1	-1	1
-1	1	1
-1	-1	-1
1	1	1

$y = \text{sign}(w^T x + b)$

① $w = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$ $b = 1$

② $w = \begin{pmatrix} \frac{1}{2} \\ 1 \end{pmatrix}$ $b = \frac{1}{2}$



linearly separable ✓

b) XOR

1	-1	1
-1	1	1
-1	-1	-1
1	1	-1



Since XOR is not linearly separable, no perceptron exists.

$h_\theta(x_n) = \sigma(a_n) = \frac{1}{1+e^{-a_n}}$

$\frac{d\,\sigma(a_n)}{d(a_n)} = -(1+e^{-a_n})^{-2}(-e^{-a_n}) = \frac{e^{-a_n}}{(1+e^{-a_n})^2}$

$= \sigma(a_n)(1-\sigma(a_n))$

$a_n = \sum_{j=0}^{d} \theta_j x_{nj}$

2) $\frac{\partial J}{\partial \theta_j} = \frac{\partial J}{\partial h_\theta(x_n)} \cdot \frac{\partial h_\theta(x_n)}{\partial a_n} \cdot \frac{\partial a_n}{\partial \theta_j} =$

$\sum_{n=1}^{N} \left(\frac{-y_n}{h_\theta(x_n)} + \frac{1-y_n}{1-h_\theta(x_n)} \right) \cdot \left(h_\theta(x_n)(1-h_\theta(x_n)) \right) \cdot x_{nj} =$

$\sum_{n=1}^{N} \left(\frac{-y_n + y_n h_\theta(x_n) + h_\theta(x_n) - y_n h_\theta(x_n)}{h_\theta(x_n)(1-h_\theta(x_n))} \cdot \left(h_\theta(x_n)(1-h_\theta(x_n)) \right) x_{nj} \right) =$

$\boxed{\sum_{n=1}^{N} \left[\left(h_\theta(x_n) - y_n \right) x_{nj} \right] = \frac{\partial J(\theta)}{\partial \theta_j}}$

this is an element of the outer product $x_n x_n^T$, so can substitute for final H

b) $\frac{\partial^2 J}{\partial \theta_j \partial \theta_k} = \sum_{n=1}^{N} \frac{\partial}{\partial \theta_k} \left[h_\theta(x_n) - y_n \right) x_{nj}] = \sum_{n=1}^{N} h_\theta(x_n)(1-h_\theta(x_n)) x_{nj} x_{nk}$

(calc der above)

$\boxed{H = \sum_{n=1}^{N} h_\theta(x_n)(1-h_\theta(x_n)) x_n x_n^T}$

For the $(j,k)^{th}$ element, $H_{j,k}$ is above.

$H_{j,k} = \frac{\partial^2 J}{\partial \theta_j \partial \theta_k}$ □

c) $z^T H z = z^T \left(\sum_{n=1}^{N} h_\theta(x_n)(1-h_\theta(x_n)) x_n x_n^T \right) z = \sum_{n=1}^{N} h_\theta(x_n)(1-h_\theta(x_n)) z^T x_n x_n^T z =$

$= \sum_{n=1}^{N} h_\theta(x_n)(1-h_\theta(x_n)) (x_n^T z)^2$

$h_\theta(x_n)(1-h_\theta(x_n))$: always ≥ 0 b/c sigmoid fn is bounded by 0,1

$(x_n^T z)^2$: always ≥ 0

$H \succeq 0$ and is PSD, so J is convex.

We know it's PSD since $z^T H z \equiv \sum_{j,k} z_j z_k H_{jk} \geq 0$

3) a) $L(\theta) = P(X_1 \dots X_n | \theta) = \theta^K (1-\theta)^{n-K}$ Let K be number of successes, or is in the data.

NO, the likelihood function does not depend on the order in which the random variables are observed, since they are i.i.d. It only matters how many successes ($X_i = 1$) K, and fails ($X_i = 0$), n-K, rather than order.

b) $\ell(\theta) = \log(L(\theta)) = \log(\theta^K (1-\theta)^{n-K}) = K \log\theta + (n-K)\log(1-\theta)$

$$\frac{d\ell(\theta)}{d\theta} = \frac{K}{\theta} + (n-K)\left(\frac{1}{1-\theta}\right)(-1) = \frac{K}{\theta} - \frac{n-K}{1-\theta}$$

$$\frac{K}{\theta} - \frac{n-K}{1-\theta} = 0$$

$$\frac{K}{\theta} = \frac{n-K}{1-\theta}$$

$$K - K\theta = n\theta - K\theta$$

$$K = n\theta$$

$$\frac{K}{n} = \theta$$

MLE ↑

$$\frac{d^2\ell(\theta)}{d\theta} = \frac{d}{d\theta}\left(\frac{K}{\theta} - \frac{n-K}{1-\theta}\right) =$$

$$\frac{-K}{\theta^2} - \frac{n-K}{(1-\theta)^2}$$

numerator always (-)
denom always (+)
Since $\frac{d^2\ell(\theta)}{d\theta}$ is < 0, function is concave down.
The critical point is a maximum. □

3 Maximum Likelihood Estimation [15 pts]

Suppose we observe the values of n independent random variables $X_1, \ldots, X_n$ drawn from the same Bernoulli distribution with parameter θ[1]. In other words, for each X_i, we know that

$$P(X_i = 1) = \theta \quad \text{and} \quad P(X_i = 0) = 1 - \theta.$$

Our goal is to estimate the value of θ from these observed values of X_1 through X_n.

For any hypothetical value $\hat{\theta}$, we can compute the probability of observing the outcome $X_1, \ldots, X_n$ if the true parameter value θ were equal to $\hat{\theta}$. This probability of the observed data is often called the *likelihood*, and the function $L(\theta)$ that maps each θ to the corresponding likelihood is called the *likelihood function*. A natural way to estimate the unknown parameter θ is to choose the θ that maximizes the likelihood function. Formally,

$$\hat{\theta}_{MLE} = \arg\max_{\theta} L(\theta).$$

(a) **(3 pts)** Write a formula for the likelihood function, $L(\theta) = P(X_1, \ldots, X_n; \theta)$. Your function should depend on the random variables $X_1, \ldots, X_n$ and the hypothetical parameter θ. Does the likelihood function depend on the order in which the random variables are observed ? **Solution:** On Paper

(b) **(6 pts)** Since the log function is increasing, the θ that maximizes the *log likelihood* $\ell(\theta) = \log(L(\theta))$ is the same as the θ that maximizes the likelihood. Find $\ell(\theta)$ and its first and second derivatives, and use these to find a closed-form formula for the MLE. **Solution:** On Paper

(c) **(3 pts)** Suppose that $n = 10$ and the data set contains six 1s and four 0s. Write a short program `likelihood.py` that plots the likelihood function of this data for each value of $\hat{\theta}$ in $\{0, 0.01, 0.02, \ldots, 1.0\}$ (use `np.linspace(...)` to generate this spacing). For the plot, the x-axis should be θ and the y-axis $L(\theta)$. Scale your y-axis so that you can see some variation in its value. Include the plot in your writeup (there is no need to submit your code). Estimate $\hat{\theta}_{MLE}$ by marking on the x-axis the value of $\hat{\theta}$ that maximizes the likelihood. Does the answer agree with the closed form answer ? **Solution:**

[1]This is a common assumption for sampling data. So we will denote this assumption as iid, short for Independent and Identically Distributed, meaning that each random variable has the same distribution and is drawn independent of all the other random variables

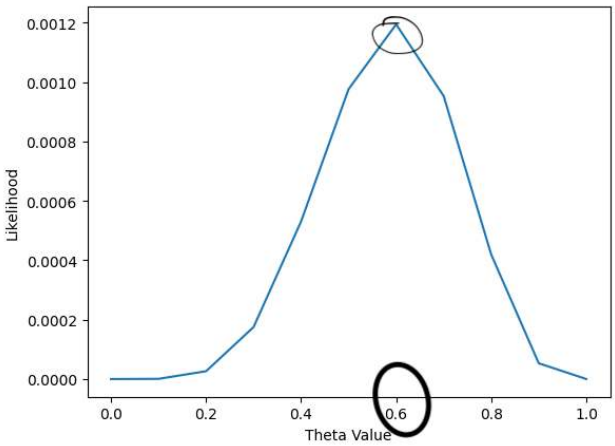


Figure 1: Part C (N = 10, K = 6)

This matches my solution from computation. MLE = k/n = 6/10, which is the same theta where likelihood is maximized in the graph.

(d) **(3 pts)** Create three more likelihood plots: one where $n = 5$ and the data set contains three 1s and two 0s; one where $n = 100$ and the data set contains sixty 1s and forty 0s; and one where $n = 10$ and there are five 1s and five 0s. Include these plots in your writeup, and describe how the likelihood functions and maximum likelihood estimates compare for the different data sets. **Solution:**

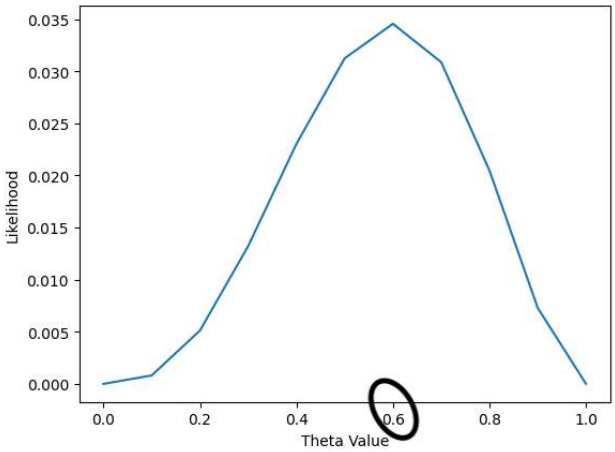


Figure 2: (N = 5, K = 3)

This says 0.6 is the MLE, and my formula calculates k/n = 3/5 = 0.6. The likelihood function range is more broad than what we saw above.

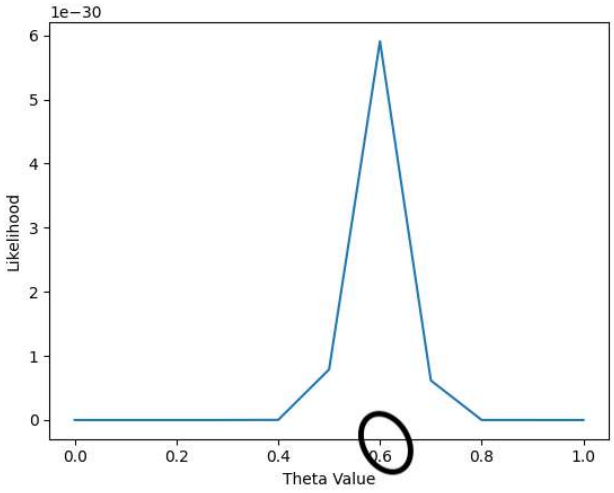


Figure 3: (N = 100, K = 60)

This matches my solution from computation. MLE = k/n = 60/100, which is the same theta where likelihood is maximized in the graph. This is the most narrow likelihood graph yet, and has less variation across the theta values.

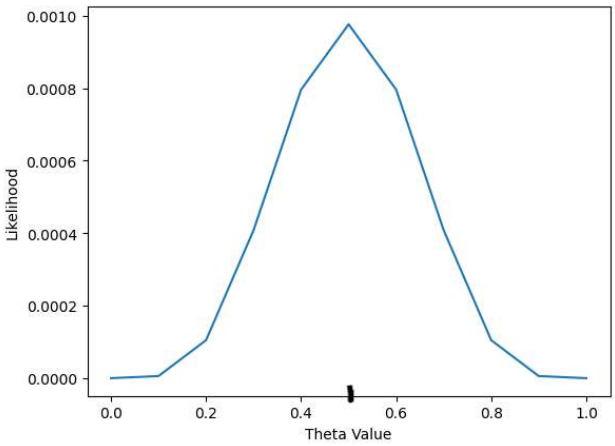


Figure 4: (N = 10, K = 5)

This shows the MLE as 0.5, which is equal to k/n, or 5/10. The looks very similar to the first graph where N=10 and K=6, but just shifts over by 0.1 on the x-axis.

4 Implementation: Polynomial Regression [40 pts]

In this exercise, you will work through linear and polynomial regression. Our data consists of inputs $x_n \in \mathbb{R}$ and outputs $y_n \in \mathbb{R}, n \in \{1, \ldots, N\}$, which are related through a target function $y = f(x)$. Your goal is to learn a linear predictor $h_{\boldsymbol{\theta}}(x)$ that best approximates $f(x)$. But this time, rather than using `scikit-learn`, we will further open the "black-box", and you will implement the regression model!

---

code and data

- code : `CS146_Winter2025_PS2.ipynb`
- data : `regression_train.csv`, `regression_test.csv`

Please use your *@g.ucla.edu* email id to access the code and data. Similar to *HW-1*, copy the colab notebook to your drive and make the changes. Mount the drive appropriately and copy the shared data folder to your drive to access via colab. The notebook has marked blocks where you need to code.

========= TODO : START =========

========= TODO : END ==========

Note: For the questions requiring you to complete a piece of code, you are expected to copy-paste your code as a part of the solution in the submission pdf. Tip: If you are using LaTeX, check out the Minted package (example) for code highlighting.

This is likely the first time that many of you are working with `numpy` and matrix operations within a programming environment. For the uninitiated, you may find it useful to work through a `numpy` tutorial first.[2] Here are some things to keep in mind as you complete this problem:

- If you are seeing many errors at runtime, inspect your matrix operations to make sure that you are adding and multiplying matrices of compatible dimensions. Printing the dimensions of variables with the `X.shape` command will help you debug.
- When working with `numpy` arrays, remember that `numpy` interprets the `*` operator as element-wise multiplication. This is a common source of size incompatibility errors. If you want matrix multiplication, you need to use the `dot` function in Python. For example, `A*B` does element-wise multiplication while `dot(A,B)` does a matrix multiply.
- Be careful when handling `numpy` vectors (rank-1 arrays): the vector shapes $1 \times N$, $N \times 1$, and N are all different things. For these dimensions, we follow the the conventions of `scikit-learn`'s `LinearRegression` class[3]. Most importantly, unless otherwise indicated (in the code documentation), both column and row vectors are rank-1 arrays of shape N, not rank-2 arrays of shape $N \times 1$ or shape $1 \times N$.

Visualization [2 pts]

[2] Try out `SciPy`'s tutorial (http://wiki.scipy.org/Tentative_NumPy_Tutorial), or use your favorite search engine to find an alternative. Those familiar with Matlab may find the "Numpy for Matlab Users" documentation (http://wiki.scipy.org/NumPy_for_Matlab_Users) more helpful.

[3] http://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LinearRegression.html

It is often useful to understand the data through visualizations. For this data set, you can use a scatter plot to visualize the data since it has only two properties to plot (x and y).

(a) Visualize the training and test data using the `plot_data(...)` function. What do you observe? For example, can you make an educated guess on the effectiveness of linear regression in predicting the data? [2 pts]

Solution:

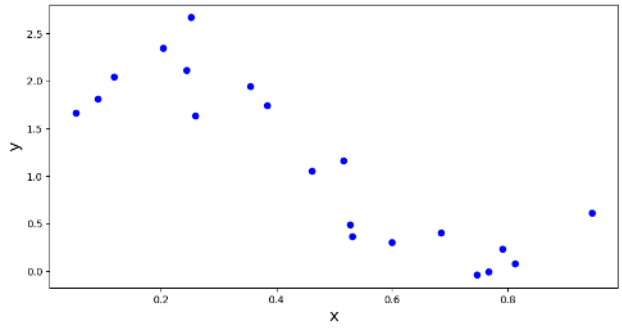


Figure 5: Train Data Plot

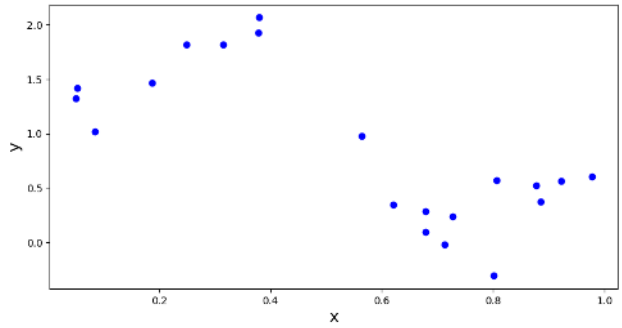


Figure 6: Test Data Plot

It does not seem like a line can demonstrate the correlation trends very well in the x/y graphs above. The data seems like it would be better suited by a non-linear model. I don't think linear regression will be too effective in predicting the data. As a result, the error will probably be high.

Linear Regression [23 pts]

Recall that linear regression attempts to minimize the objective function

$$J(\boldsymbol{\theta}) = \sum_{n=1}^{N} (h_{\boldsymbol{\theta}}(\boldsymbol{x}_n) - y_n)^2.$$

In this problem, we will use the matrix-vector form where

$$\boldsymbol{y} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{pmatrix}, \qquad \boldsymbol{X} = \begin{pmatrix} \boldsymbol{x}_1^T \\ \boldsymbol{x}_2^T \\ \vdots \\ \boldsymbol{x}_N^T \end{pmatrix}, \qquad \boldsymbol{\theta} = \begin{pmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \\ \vdots \\ \theta_D \end{pmatrix}$$

and each instance $\boldsymbol{x}_n = \big(1, x_{n,1}, \ldots, x_{n,D}\big)^T$.

In this instance, the number of input features $D = 1$.

Rather than working with this fully generalized, multivariate case, let us start by considering a simple linear regression model:

$$h_{\boldsymbol{\theta}}(\boldsymbol{x}) = \boldsymbol{\theta}^T \boldsymbol{x} = \theta_0 + \theta_1 x_1$$

The Colab notebook contains the skeleton code for the class `PolynomialRegression`. Objects of this class can be instantiated as `model = PolynomialRegression (m)` where m is the degree of the polynomial feature vector where the feature vector for instance n, $\big(1, x_{n,1}, x_{n,1}^2, \ldots, x_{n,1}^m\big)^T$. Setting $m = 1$ instantiates an object where the feature vector for instance n, $\big(1, x_{n,1}\big)^T$.

(b) Note that to take into account the intercept term (w_0), we can add an additional "feature" to each instance and set it to one, e.g. $x_{i,0} = 1$. This is equivalent to adding an additional first column to $\boldsymbol{X}$ and setting it to all ones [2 pts].

Modify `PolynomialRegression.generate_polynomial_features(...)` to create the matrix $\boldsymbol{X}$ for a simple linear model.

Solution:

```
### ========== TODO : START ========== ###
# part b: modify to create matrix for simple linear model
# part g: modify to create matrix for polynomial model
Phi = np.concatenate((np.ones((X.shape[0],1)),X),axis=1)
m = self.m_

### ========== TODO : END ========== ###

return Phi
```

Figure 7: Part B

(c) Before tackling the harder problem of training the regression model, complete `PolynomialRegression.predict(...)` to predict $\boldsymbol{y}$ from $\boldsymbol{X}$ and $\boldsymbol{\theta}$. [3 pts]

Solution:

```
### ========== TODO : START ========== ###
# part c: predict y
y = np.dot(X, self.coef_)
### ========== TODO : END ========== ###
```

Figure 8: Part C

(d) One way to solve linear regression is through gradient descent (GD).

Recall that the parameters of our model are the θ_j values. These are the values we will adjust to minimize $J(\boldsymbol{\theta})$.

$$J(\boldsymbol{\theta}) = \sum_{n=1}^{N} (h_{\boldsymbol{\theta}}(\boldsymbol{x}_n) - y_n)^2$$

In gradient descent, each iteration performs the update

$$\theta_j \leftarrow \theta_j - 2\alpha \sum_{n=1}^{N} (h_{\boldsymbol{\theta}}(\boldsymbol{x}_n) - y_n)\, x_{n,j} \quad \text{(simultaneously update } \theta_j \text{ for all } j\text{)}.$$

With each step of gradient descent, we expect our updated parameters w_j to come closer to the parameters that will achieve the lowest value of $J(\boldsymbol{\theta})$. [10 pts]

- **(2 pts)** As we perform gradient descent, it is helpful to monitor the convergence by computing the cost, *i.e.*, the value of the objective function J. Complete `PolynomialRegression.cost(...)` to calculate $J(\boldsymbol{\theta})$.

  If you have implemented everything correctly, then the following code snippet should return 40.234.

  Solution:

```
### ========== TODO : START ========== ###
# part d: compute J(theta)
cost = np.sum(np.power(self.predict(X) - y, 2))
### ========== TODO : END ========== ###
```

Figure 9: Part D cost

- **(3 pts)** Next, implement the gradient descent step in `PolynomialRegression.fit_GD(...)`. The loop structure has been written for you, and you only need to supply the updates to $\boldsymbol{\theta}$ and the new predictions $\hat{y} = h_{\boldsymbol{\theta}}(\boldsymbol{x})$ within each iteration.

  We will use the following specifications for the gradient descent algorithm:

  - We run the algorithm for $10,000$ iterations.
  - We terminate the algorithm earlier if the value of the objective function is unchanged across consecutive iterations.
  - We will use a fixed learning rate.

  Solution:

```
### ========== TODO : START ========== ###
# part d: update theta (self.coef_) using one step of GD
# hint: you can write simultaneously update all theta using vector math

sum = np.dot(X.T,np.dot(X,self.coef_)-y)
self.coef_ -= 2*eta*sum
# track error
# hint: you cannot use self.predict(...) to make the predictions
y_pred = np.dot(X, self.coef_)
err_list[t] = np.sum(np.power(y - y_pred, 2)) / float(n)
### ========== TODO : END ========== ###
```

Figure 10: Part D fit GD

- **(5 pts)** Experiment with different values of learning rate $\eta = 10^{-6}$, 10^{-5}, 10^{-3}, 0.1 and make a table of the coefficients, number of iterations until convergence (this number will be 10,000 if the algorithm did not converge in a smaller number of iterations) and the final value of the objective function. How do the coefficients compare? How quickly does each algorithm converge? Do you observe something strange when you run with $\eta = 0.1$? Explain the observation and causes.

  Solution:

Learning Rate	Coefficients	Number of Iterations	Cost	Time
$\eta = 10^{-6}$	0.36400847, 0.09215787	10000	25.863	0.207
$\eta = 10^{-5}$	1.15699657, -0.22522908	10000	13.159	0.198
$\eta = 10^{-3}$	2.4464068, -2.816353	7020	3.913	0.159
$\eta = 0.1$	NaN	10000	NaN	0.197

Table 1: Step Size Results

The larger the step size got, the wider the range of coefficients became. For example, in the first two the coefficients are much closer than in the third row, where the algorithm did converge, so it makes sense that the coefficients were getting larger and smaller, towards that respectively. In all but the third step size, the algorithm used the max number of iterations, meaning it could not find a minimum. In the third row, the algorithm was able to find a minimum and converge. This makes sense since it had a lower cost. The one that did converge took the least time, which makes sense since it used less iterations. When I ran it with eta = 0.1, I got an error that there was overflow and therefore it could not complete the expected calculations. A possible cause is that the step size results in updates that were too large, and it could not finish the math.

(e) In class, we learned that the closed-form solution to linear regression is

$$\boldsymbol{\theta} = (\boldsymbol{X}^T\boldsymbol{X})^{-1}\boldsymbol{X}^T\boldsymbol{y}.$$

Using this formula, you will get an exact solution in one calculation: there is no "loop until convergence" like in gradient descent. [4 pts]

- **(2 pts)** Implement the closed-form solution `PolynomialRegression.fit(...)`.

  Solution:

```
### ========== TODO : START ========== ###
# part e: implement closed-form solution
# hint: use np.dot(...) and np.linalg.pinv(...)
#       be sure to update self.coef_ with your solution

theta = np.dot(np.dot(np.linalg.pinv(np.dot(X.T,X)),X.T),y)
self.coef_ = theta
### ========== TODO : END ========== ###
```

Figure 11: Part E

- **(2 pts)** What is the closed-form solution coefficients? How do the coefficients and the cost compare to those obtained by GD? How quickly does the algorithm run compared to GD?

Solution: The coefficients are 2.44640709, and -2.81635359, the cost was 3.913 and the time was 0.002848 seconds. This matched the coefficients and cost found above in row 3. Compared to the GD step size run that had the same coefficients and cost, this one ran in much less time (as opposed to 0.159). It was much faster.

(f) Finally, set a learning rate η for GD that is a function of k (the number of iterations) (use $\eta_k = \frac{1}{1+k}$) and converges to the same solution yielded by the closed-form optimization (minus possible rounding errors). Update `PolynomialRegression.fit_GD(...)` with your proposed learning rate. How many iterations does it take the algorithm to converge with your proposed learning rate? [4 pts]

Solution:

```
### ========== TODO : START ========== ###
# part f: update step size
# change the default eta in the function signature to 'eta=None'
# and update the line below to your learning rate function
if eta_input is None :
    eta = 1/(1+t) # change this line
else :
    eta = eta_input
### ========== TODO : END ========== ###
```

Figure 12: Part F

To get to the same coefficients and cost, it took 1718 iterations and 0.057 seconds.

Polynomial Regression[15 pts]

Now let us consider the more complicated case of polynomial regression, where our hypothesis is

$$h_{\boldsymbol{\theta}}(\boldsymbol{x}) = \boldsymbol{\theta}^T \phi(\boldsymbol{x}) = \theta_0 + \theta_1 x + \theta_2 x^2 + \ldots + \theta^m x^m.$$

(g) Recall that polynomial regression can be considered as an extension of linear regression in which we replace our input matrix $\boldsymbol{X}$ with

$$\boldsymbol{\Phi} = \begin{pmatrix} \phi(x_1)^T \\ \phi(x_2)^T \\ \vdots \\ \phi(x_N)^T \end{pmatrix},$$

where $\phi(x)$ is a function such that $\phi_j(x) = x^j$ for $j = 0, \ldots, m$.

Update `PolynomialRegression.generate_polynomial_features(...)` to create an $m+1$ dimensional feature vector for each instance. [4 pts]

Solution:

```
### ========== TODO : START ========== ###
# part b: modify to create matrix for simple linear model
# part g: modify to create matrix for polynomial model
Phi = np.concatenate((np.ones((X.shape[0],1)),X),axis=1)

for i in range(2,self.m_+1):
    Phi = np.concatenate((Phi,np.power(X,i)),axis=1)
m = self.m_

### ========== TODO : END ========== ###
```

Figure 13: Part G

(h) Given N training instances, it is always possible to obtain a "perfect fit" (a fit in which all the data points are exactly predicted) by setting the degree of the regression to $N-1$. Of course, we would expect such a fit to generalize poorly. In the remainder of this problem, you will investigate the problem of overfitting as a function of the degree of the polynomial, m. To measure overfitting, we will use the Root-Mean-Square (RMS) error, defined as

$$E_{RMS} = \sqrt{J(\boldsymbol{\theta})/N},$$

where N is the number of instances.[4]

Why do you think we might prefer RMSE as a metric over $J(\boldsymbol{\theta})$?

Implement `PolynomialRegression.rms_error(...)`. [4 pts]

Solution: We might prefer RMSE as a metric over the other cost function because it takes 'N,' the number of instances into account. This makes it easier to compare models trained on datasets of different sizes by accounting for N due to the normalization. In other words, we are seeing the average error per data point. Also, increasing N, or having more data lowers the likelihood that we are overfitting and decreasing N, or having less data, increases the chance of overfitting–this function is a little more intuitive.

```
### ========== TODO : START ========== ###
# part h: compute RMSE
error = np.sqrt(self.cost(X,y)/X.shape[0])
### ========== TODO : END ========== ###
```

Figure 14: Part H

(i) For $m = 0, \ldots, 10$ (where m is the order of the polynomial transformation applied to features), use the closed-form solver to determine the best-fit polynomial regression model on the training data, and with this model, calculate the RMSE on both the training data and the test data. Generate a plot depicting how RMSE varies with model complexity (polynomial degree) – you should generate a single plot using `plot_erms(...)` function , and include this plot in your writeup. Which degree polynomial would you say best fits the data? Was there evidence of under/overfitting the data? Use your plot to justify your answer. [7 pts]

Solution:

[4] Note that the RMSE as defined is a biased estimator. To obtain an unbiased estimator, we would have to divide by $n-k$, where k is the number of parameters fitted (including the constant), so here, $k = m+1$.

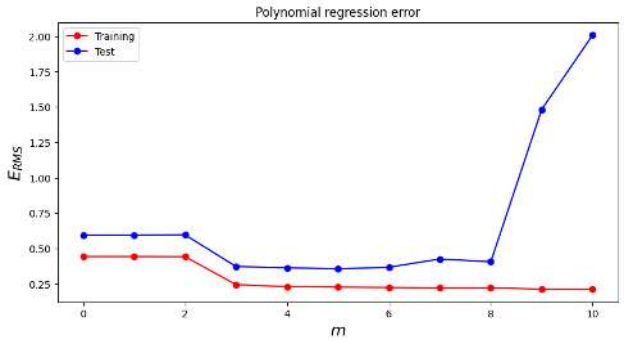


Figure 15: Part I

I would say a 4 or 5 degree polynomial best fits my data. This is where we see both errors being relatively small (compared to the other degrees), but not so far apart that only train error is very low and there is overfitting. After 5, test error starts to go up. There is evidence of overfitting towards the high degrees (8-10) since the test error spikes up while the train error is at its lowest value on the graph.